Entregable Semana 7 (AVZ7)

Low Level Software Design de R1 y R2

Grupo 3 — Horario H0892

TEL354: Redes Definidas por Software — 2026-2

10 de octubre de 2026

Coach	Germán Espinoza
Profesor	Christian Isaac Quispe Ordoñez
Integrantes	Rojas Soplin, Rodrigo Franco — 20213686 (líder) Oropeza León, Piero Jesús — 20212733 (arquitecto) Altamirano Ramírez, Camila Nicole — 20220964 (PM / QA) Tavella Rivero, Fernando Vincenzo — 20182693 (DevOps / SRE)
Archivo Paideia	AVZ7-TEL354_2026-2_G3

Alcance de este documento

Diseño de bajo nivel (LLSD) de **R1 (control de acceso según rol)** y **R2 (recursos privilegiados)**: estructura interna de cada servicio que interviene, funciones y algoritmos, estructuras de datos, contratos de interfaz campo a campo y las entradas OpenFlow exactas que se instalan en los switches. El nivel de detalle busca que cada integrante pueda pasar a código sin volver a decidir lo que aquí queda fijado.

1 Punto de partida y convenciones

1.1 Fuentes y regla de precedencia

- **Repositorio SDN-project, fase G (LLD)** (commit 9626b2b): APIs, eventos, datos, configuraciones, reglas y pseudocódigo.
- **Laboratorio 4** (northbound REST en Python): lecciones de confirmación por lote y de granularidad por servicio.

1.2 Dentro y fuera del alcance

- **Dentro:** M06 Portal (.5) e IAM/AAA (.10), M07 Registro (.11), M05 Políticas (.15), la app G3 del clúster ONOS (M01), M10 Consola (.17) en lo que toca a elevaciones, el pipeline de reglas de R1/R2 en los switches (M11) y la configuración de FreeRADIUS (.23) y OpenLDAP (.24).
- **Frontera:** R4 (Monitor, Detección, Incidentes) solo aparece como la prioridad 1000 del pipeline y la orden `APLICAR_MITIGACION`. M08 Auditoría solo como consumidor de eventos.
- **Convenciones:** servicios en **Python 3.12** (FastAPI, SQLAlchemy, `pika`, `pyrad`); app G3 en **Java 11** sobre ONOS 2.7 (OSGi, JAX-RS). Toda cifra es **valor inicial o presupuesto de diseño**; este documento no contiene mediciones: fija qué se instala y qué se puede medir.

2 Vista de módulos de R1 y R2

Hay un solo punto de decisión, **Políticas (M05)**, que es además el único que emite órdenes a la red por I2. Hay un solo traductor a OpenFlow, la **app G3** del clúster ONOS. El resto identifica (Portal, IAM), aporta atributos (Registro) o es la interfaz humana (Consola). Los servicios no se hablan por llamadas internas: lo síncrono va por REST y lo que informa va por AMQP.

Cód.	Servicio	Dónde (VLAN 20)	Pregunta que responde	Interfaces que usa	Estado
M01	Clúster ONOS 2.7 + app G3	ONOS-1 10.0.0.20 · ONOS-2 .30 · ONOS-3 .40	¿Cómo se escribe la decisión en el switch?	I1 OpenFlow 1.3 TCP 6653 · I2 REST/HTTPS TCP 8181 (servidor) · I9 Atomix TCP 5679/9876	ConsistentMap en Atomix
M06	Portal cautivo	10.0.0.5	¿Por dónde se acredita?	HTTP 80 (redirección) · HTTPS 443 · I3 hacia el IAM	Sin estado
M06	IAM/AAA	10.0.0.10	¿Quién es? ¿Con qué rol?	I3 · I4 RADIUS UDP 1812/1813 · I2 (consulta de asociaciones) · I5 · I6 · I7	`bd_iam`
M07	Registro de dispositivos	10.0.0.11	¿Puede este equipo intentar un rol privilegiado?	I7 REST · I5 · I6	`bd_registro`
M05	Políticas (PDP)	10.0.0.15	¿Qué puede hacer, aquí y ahora?	I2 cliente (único) · I5 · I6 · I7	`bd_politicas`
M10	Consola	10.0.0.17	¿Cómo opera y aprueba el humano?	HTTPS 443 · I7 · I5 (alertas)	Sin estado propio
M08	Auditoría	10.0.0.16	¿Quién hizo qué y cuándo?	I5 (cola `auditoria`) · I6	`bd_auditoria` + JSONL encadenado
M09	RabbitMQ	10.0.0.21	Bus de eventos	AMQP 0-9-1 TCP 5672	Colas durables + DLQ
—	PostgreSQL · FreeRADIUS · OpenLDAP · DNS · chrony	.22 · .23 · .24 · .28 · .25	Soporte	SQL 5432 · RADIUS 1812/1813 · LDAP 389 · DNS 53 · NTP 123	—
M11	Switches Pica8 (10)	C1, C2, NC1, NC2, D1, D2, A1–A4	Ejecutan y cuentan	I1	Tablas de flujo

Tabla 2.1 — Módulos que intervienen en R1 y R2, con el código del catálogo y la dirección de la v2.

2.1 Estructura interna común de los servicios

Todos los servicios Python repiten la misma estructura en capas, que es la de G-07: `api` (entrada síncrona), `dominio` (lógica pura, sin E/S, que es lo que se prueba con unit testing), `repositorio` (su base y solo la suya) y `eventos` (publicador/consumidor AMQP). Cada servicio expone `/health/live` y `/health/ready`; si el servicio no está *ready* no recibe tráfico, pero la red sigue operando con lo ya instalado.

Código: `servicios/{comun,portal,iam,registro,politicas,consola}` en Python y `onos-app-g3` (paquete `org.pucp.tel354.g3`: `api/` con `OrdenesResource`, `impl/` con `TraductorPerfil`, `RegistroCookies`, `GestorInactividad`, `RelayVencimientos`, `RedireccionPortal`, `InstaladorEsqueleto`, y `store/` con los ConsistentMap `g3-*`). `org.onosproject.fwd` permanece desactivado.

3 Interfaces y protocolos que usa R1/R2

ID	Entre	Protocolo · puerto	Sincronía · tiempo límite	En R1/R2 transporta
I1	ONOS ↔ switches	OpenFlow 1.3 · TCP 6653 · diseño in-band (VLAN 20, p65000) · prototipo por `ma1`	Mixta; `ECHO` 5 s, caído a los 3 latidos	FLOW_MOD, BARRIER, FLOW_REMOVED, PACKET_IN/OUT, MULTIPART
I2	Políticas, IAM → app G3	REST/HTTPS · JSON · TCP 8181 · `/api/v1/...` · clave de servicio por cliente	Síncrona; 2 s por nodo, failover a los otros 2	`APLICAR_PERFIL`, `RETIRAR`, asociaciones
I3	Navegador → Portal → IAM	HTTPS 443 (formularios) → REST/JSON	Síncrona; 3 s para la respuesta del login	Credenciales, TOTP, solicitud de elevación
I4	IAM → FreeRADIUS → OpenLDAP	RADIUS UDP 1812/1813 → LDAP bind TCP 389	Síncrona; 3 s × 2 intentos	Access-Request/Accept/Reject, Class, Accounting
I5	Servicio ↔ servicio	AMQP 0-9-1 · RabbitMQ TCP 5672 · exchange `plataforma` (topic)	Asíncrona; al menos una vez	SessionOpened, ProfileApplied, Elevation*, …
I6	Servicio → su base	SQL · PostgreSQL TCP 5432	Síncrona local	Sesiones, perfiles, elevaciones, dispositivos
I7	Consola → servicios	REST/HTTPS · JSON · cookie de sesión del operador	Síncrona	Aprobaciones, registro de dispositivos, perfiles
I9	ONOS ↔ ONOS	Atomix/Raft TCP 5679 · mensajería de clúster TCP 9876	Consenso	Cookies, asociaciones y suscripciones replicadas

Tabla 3.1 — Subconjunto de la tabla I1–I9 de la v2 que usan R1 y R2. I8 (contadores) solo interviene en R4.

Regla de diseño heredada: lo que exige confirmación viaja por **REST síncrono (I2)** y su eco se publica por **AMQP (I5)**. Ningún servicio salvo Políticas ordena a la red; el IAM solo usa I2 para **consultar** la asociación MAC ↔ switch:puerto ↔ IP.

4 Perfiles, recursos y matriz de acceso

4.1 Escalera de perfiles

Perfil	Cómo se obtiene	Vigencia (idle · tope)	Atributo `Class` / rol	Cookie
BASE	Presencia física: la app G3 aprende MAC, puerto e IP	Permanente mientras el equipo esté	—	`ESQ-…`
ACADÉMICO	Login en el portal → RADIUS/LDAP	30 min · 12 h	`ACAD.ALUMNO` · `ACAD.DOCENTE` · `ACAD.AUTORIDAD`	`SES-…`
ELEVADO	Solicitud del académico + aprobación de un Admin con TOTP	— · 1 h (TTL)	Se apoya en una sesión ACADÉMICA activa	`ELE-…`
OPERADOR	Dispositivo registrado + usuario/contraseña (bcrypt) + TOTP	15 min · 8 h	`OP.TI` · `OP.ADMIN` · `OP.SUPERADMIN`	`SES-…`

Tabla 4.1 — Cada peldaño exige más identidad y dura menos (v2, Fig. 11). Hay 8 roles efectivos en 4 niveles jerárquicos.

Los **subperfiles académicos** comparten el mismo alcance de red: la diferencia es la **elegibilidad para elevarse**. Como valor inicial, docente y autoridad son elegibles para el recurso privilegiado y el alumno no; se configura en `perfiles.elegible_elevacion` (hueco H5). Así el número de subperfiles no multiplica las reglas en el switch.

4.2 Inventario y clasificación de recursos

Recurso	Dirección · servicio	Clase	Protección aplicada
Portal, DNS, DHCP	10.0.0.5:443 · 10.0.0.28:53 · UDP 67/68	Base	Alcanzables por todos, incluso en BASE (p150/p140)
Servidor general	10.2.0.10:443	General (mínima)	Cualquier sesión ACADÉMICA u OPERADOR
Salida externa	10.3.0.0/24 (prototipo, VLAN 40) · Internet en el campus	General	Sesión ACADÉMICA u OPERADOR
Servidor privilegiado (laboratorio / investigación)	10.2.0.11:22 y :443	Privilegiado (intermedia)	Solo con elevación aprobada y TTL de 1 h, o rol Admin/Superadmin
Servidor crítico (BD institucional)	10.2.0.12:5432	Crítico	Solo Superadmin, con TOTP y desde un dispositivo registrado
Consola y Grafana	10.0.0.17:443 · 10.0.0.26:3000	Gestión	Operadores TI, Admin y Superadmin
Controladores ONOS	10.0.0.20/.30/.40:8181	Crítico (infraestructura)	Admin y Superadmin
IAM, Políticas, broker, BD, RADIUS, LDAP	10.0.0.10–.24	Crítico (infraestructura)	Nadie desde el plano de datos: solo entre servicios de VLAN 20
Switches (gestión)	`ma1`, red del laboratorio	Crítico	Fuera del plano de datos (canal OOB)

Tabla 4.2 — Recursos de R2 con su clase. Son las tres clases de la v2 (Fig. 13) más la infraestructura de la propia solución.

4.3 Matriz perfil × recurso

Recurso	BASE	ACAD	ELEVADO	OP.TI	OP.ADMIN	OP.SA
Portal · DNS · DHCP	Sí	Sí	Sí	Sí	Sí	Sí
Srv general 10.2.0.10:443	No	Sí	Sí	Sí	Sí	Sí
Salida externa	No	Sí	Sí	Sí	Sí	Sí
Srv privilegiado 10.2.0.11:22/443	No	No	Sí (TTL)	No	Sí	Sí
Srv crítico 10.2.0.12:5432	No	No	No	No	No	Sí
Consola .17:443 · Grafana .26:3000	No	No	No	Sí	Sí	Sí
ONOS .20/.30/.40:8181	No	No	No	No	Sí	Sí
Servicios internos (.10–.24)	No	No	No	No	No	No
Otro host de usuarios (este-oeste)	No	No	No	No	No	No

Tabla 4.3 — Matriz de la política. Cada «Sí» fuera de la fila base es una entrada p200 de la sesión; todo lo demás cae en el deny del dispositivo (p105) dentro del ASIC.

5 Pipeline del switch: entradas OpenFlow

La jerarquía de prioridades es una jerarquía de autoridad y es la misma en toda la red: **mitigación > sesión > esqueleto > caminos**. La política se aplica en el **switch de ingreso**; los tramos siguientes solo resuelven caminos.

5.1 Entradas del switch de acceso

Prio.	Entrada	Match	Acción	Ámbito · cookie	Vigencia
65000	Canal de control (solo diseño in-band)	`tcp,tp=6653` ↔ ONOS .20/.30/.40	OUTPUT hacia C1/C2 (cola estricta, DSCP CS6)	Por switch · `ESQ`	Permanente
1000	Mitigación R4 (frontera)	origen → objetivo	`meter:n` → salida / DROP	Por incidente · `INC`	`hard = ttl`
200	Sesión (ACAD / OP)	`in_port, eth_src, ipv4_src + ipv4_dst [, ip_proto, tp_dst]` por destino del alcance	OUTPUT (camino calculado o grupo FF)	Por sesión · `SES`	`hard = tope`, sin `idle`
200	Elevación	Igual, con destino 10.2.0.11 y `tp_dst` 22 / 443	OUTPUT	Por elevación · `ELE`	`hard = 3600, SEND_FLOW_REM`
150	Portal	`ipv4_dst=10.0.0.5, tcp, tp_dst=443`	OUTPUT	Por switch · `ESQ`	Permanente
140	DNS y DHCP (C2)	`ipv4_dst=10.0.0.28, udp/tcp 53 · udp, tp_dst=67`	OUTPUT	Por switch · `ESQ`	Permanente

Prio.	Entrada	Match	Acción	Ámbito · cookie	Vigencia
110	Redirección al portal	`in_port, eth_src, tcp, tp_dst=80`	CONTROLLER (meter `m_redir` 20 pps)	Por dispositivo · `ESQ`	Permanente
105	Deny del dispositivo	`in_port, eth_src`	DROP (cuenta)	Por dispositivo · `ESQ`	Mientras el equipo esté
100	DROP a gestión	`ipv4_dst=10.0.0.0/24`	DROP	Por switch · `ESQ`	Permanente
30	Acceso desconocido	`in_port=<puerto de acceso>`	CONTROLLER (meter `m_miss` 50 pps)	Por puerto de acceso · `ESQ`	Permanente
20	Caminos	`ipv4_dst=<host /32>` (tránsito hacia hosts locales)	OUTPUT o grupo FAST_FAILOVER	Por destino · `CAM`	Mientras valga la topología
0	Table-miss	*	DROP	—	Permanente

Tabla 5.1 — Pipeline de ingreso. Las entradas de ARP se resuelven con el proxy ARP de la app G3 (p130 `eth_type=0x0806 → CONTROLLER`, meter `m_arp` 50 pps).

De la tabla salen cuatro propiedades que el diseño garantiza y que luego pueden comprobarse. **(i)** Un equipo sin login solo alcanza el portal, el DNS y el DHCP; su HTTP sube con meter para la redirección y todo lo demás muere en p105. **(ii)** Un usuario autenticado que intenta un recurso no permitido no encuentra p200 y cae en **su** p105: se descarta en el ASIC sin PACKET_IN. **(iii)** El anti-spoofing sale gratis: las p200 exigen el trío `in_port`, `eth_src` e `ipv4_src`, así que una IP ajena no coincide y cae en p105 (o en p30 si además la MAC es desconocida). **(iv)** La sesión nunca abre la red de gestión salvo destinos puntuales de operador, porque no existe ninguna p200 hacia 10.0.0.0/24 fuera de esos destinos.

5.2 Ejemplo: un docente en A1:p20 tras el login

```
# esqueleto: al aprender el equipo (primer ARP)
A1 p110 cookie=0x03..17 in_port=p20,dl_src=52:54:00:01:10,tcp,tp_dst=80  -> meter:2,CONTROLLER
A1 p105 cookie=0x03..17 in_port=p20,dl_src=52:54:00:01:10                -> drop
# sesion ACAD.DOCENTE SES-000123: una orden APLICAR_PERFIL = un lote + BARRIER
A1 p200 cookie=0x01..7B hard=43200 send_flow_rem
        in_port=p20,dl_src=52:54:00:01:10,nw_src=10.1.0.10,
        nw_dst=10.2.0.10,tcp,tp_dst=443                                  -> output:p1
A1 p200 cookie=0x01..7B hard=43200 send_flow_rem
        in_port=p20,dl_src=52:54:00:01:10,nw_src=10.1.0.10,nw_dst=10.3.0.0/24 -> output:p1
# elevacion ELE-000045 aprobada (TTL 1 h)
A1 p200 cookie=0x05..2D hard=3600 send_flow_rem  ...,nw_dst=10.2.0.11,tcp,tp_dst=22  -> output:p1
A1 p200 cookie=0x05..2D hard=3600 send_flow_rem  ...,nw_dst=10.2.0.11,tcp,tp_dst=443 -> output:p1
```

Figura 5.1 — Entradas reales de un docente. `output:p1` es el uplink hacia D1; con grupos fast-failover (V3) se reemplaza por `group:<id>` con vigía sobre p1 y respaldo por D2.

5.3 Cookies

Bits 63–56 (tipo)	Uso	Correlativo (bits 55–0)	Retiro
0x01 SES	Sesión ACADÉMICA u OPERADOR	Número de `SES-xxxxxx`	Por cookie: logout, inactividad, tope, MAC_Moved
0x02 INC	Mitigación R4	Número de `INC-xxxx`	TTL o retiro del incidente
0x03 ESQ	Esqueleto: genérico (correlativo 0) o por dispositivo (correlativo = id del dispositivo)	0 o id interno	Al desaparecer el dispositivo
0x04 CAM	Caminos	Id del destino	Recálculo de topología
0x05 ELE	Elevación	Número de `ELE-xxxxxx`	TTL (`hard_timeout`) o revocación
0x06 SRV	Servicio declarado (salida de servidores y plataforma)	Id del servicio	Baja del servicio

Tabla 5.2 — Esquema único de cookie. La app G3 guarda el registro texto ↔ 64 bits en el ConsistentMap `g3-cookies`. Se retira siempre por cookie y con el ApplicationId de G3, nunca reglas ajenas.

5.4 Zona C1/C2 y switches de tránsito

- **Servicios declarados (p200, `SRV`)**: una entrada por servicio, `in_port=<puerto del servicio>, ipv4_src=<IP del servicio> → ipv4_dst=10.1.0.0/24 → caminos`. Es el tráfico de vuelta de portal, DNS, Consola, Grafana, ONOS y servidores .10/.11/.12. Es *stateless*: un servidor comprometido podría iniciar conexiones hacia usuarios, un riesgo que se declara y queda para R3.

- **Segunda capa**: p100 `ipv4_src=10.1.0.0/24, ipv4_dst=10.0.0.0/24 → DROP`, con excepciones p150 (portal) y p140 (DNS) y con las p200 de sesiones de operador, que Políticas ordena también en C1/C2 (espejo de 2 a 6 entradas por operador, según el rol). Además, p100 `ipv4_src=10.1.0.0/24, ipv4_dst=10.2.0.12 → DROP` salvo la p200 del Superadmin.
- **D1, D2, NC1 y NC2** solo llevan caminos (p20) y table-miss: la política ya se aplicó en el ingreso.

5.5 Variante multi-tabla (si V1 la confirma)

Si V1 confirma que PicOS acepta varias tablas y `write_metadata` con máscara, `TraductorPerfil` pasa a T0 seguridad, T1 identidad (`metadata = perfil`), T2 autorización (`metadata × destino`) y T3 caminos, sin cambiar la orden I2. Su conveniencia depende de la capacidad de TCAM y del número de equipos por switch (H1, H2).

6 M06 Portal e IAM/AAA

El portal es solo la puerta: presenta formularios, no valida nada y no conecta nada. El IAM verifica la identidad por la ruta que corresponde: la comunidad por RADIUS contra el IdP y los operadores contra su propio repositorio con TOTP. Después crea la sesión ligada al dispositivo y publica `SessionOpened`. No escribe reglas: espera el eco `ProfileApplied` para responder «sesión activa».

6.1 Endpoints I3 (portal → IAM)

Recurso	Qué hace	Respuestas
`GET http://*/*` (redirección)	La app G3 reescribe el primer SYN a :80 de un equipo en BASE hacia 10.0.0.5:80 (par de reglas temporales, TTL 30 s). El portal responde `302 → https://10.0.0.5/login`	302
`GET /login`	Selector de población: «Comunidad universitaria» u «Operación». No concede nada	200
`POST /login` {usuario, clave, poblacion, totp?}	Verifica, liga la sesión a (MAC, switch:puerto, IP) y espera el eco de la regla (§6.3)	200 ACTIVA · 202 APLICANDO · 401 · 403 · 429 · 503
`POST /logout`	Cierra la sesión → `SessionClosed` → Políticas ordena `RETIRAR`	200
`GET/POST /elevacion`	Solicitud del académico (recurso, motivo, duración ≤ 1 h) → `ElevationRequested`	202

6.2 Clases del IAM

```
class ServicioSesiones:                       # dominio, sin E/S
    def crear(usuario, rol, asociacion, vigencia) -> Sesion
  def validar(cookie_hash, ip_origen) -> Sesion  # ip ligada, estado ACTIVA, ahora < tope
    def cerrar(sesion_id, motivo) -> Sesion              # idempotente
class VerificadorComunidad:  def verificar(usuario, clave) -> (bool, clase)   # pyrad, I4
class VerificadorOperador:  def verificar(usuario, clave, totp) -> (bool, rol) # bcrypt + RFC 6238 ±1
class Limitador:  def permitir(clave=(ip, usuario)) -> bool  # 5 fallos / 5 min -> 429
class ClienteI2:  def asociacion(ip) -> Asociacion  # GET /api/v1/asociaciones
class EsperaEcos:            # consumidor AMQP de ProfileApplied / ProfileRejected
    def esperar(sesion_id, timeout=2.5) -> Resultado        # Future por sesion_id
```

6.3 Algoritmo del login

```
async def login(req, ip_origen):
    if not limitador.permitir((ip_origen, req.usuario)): return 429
    if req.poblacion == 'operacion':
        ok, rol = verificador_op.verificar(req.usuario, req.clave, req.totp)  # nunca sin TOTP
        clase = 'OP.' + rol
    else:
        ok, clase = await verificador_com.verificar(req.usuario, req.clave)   # RADIUS 3 s x 2
    if not ok: limitador.fallo(...); publicar(LoginFailed); return 401
    asoc = await i2.asociacion(ip_origen)          # MAC, switch, puerto aprendidos por ONOS
    if asoc is None: return 403                    # IP no aprendida: posible suplantacion
    s = sesiones.crear(req.usuario, clase, asoc, VIGENCIA[clase]); repo.guardar(s)
    fut = ecos.registrar(s.id)                     # antes de publicar: evita la carrera
    publicar('SessionOpened', s)                   # -> Politicas decide y ordena por I2
    r = await fut.esperar(2.5)
    if r is APLICADO:  radius.accounting('Start', s); return 200, cookie_opaca(s)
    if r is DENEGADO:  sesiones.cerrar(s.id, r.motivo); return 403, r.motivo
    return 202, cookie_opaca(s)                    # la pagina consulta GET /sesion/estado
```

El invariante central de R1 es el de la Fig. 10 de la v2: **la respuesta «sesión activa» solo sale después de que el switch confirmó la regla** (BARRIER_REPLY → 201 → `ProfileApplied`). Si saliera antes, la primera petición del usuario caería en p105 y vería un error.

6.4 Estados de la sesión

Estado	Evento	Siguiente	Acción
—	Credenciales válidas y asociación encontrada	PENDIENTE_REGLA	Guardar, publicar `SessionOpened`
PENDIENTE_REGLA	`ProfileApplied`	ACTIVA	200 al portal; Accounting-Start
PENDIENTE_REGLA	`ProfileRejected` (DENY o error de instalación)	RECHAZADA	403; auditoría
PENDIENTE_REGLA	Sin eco en 2,5 s	PENDIENTE_REGLA	202; la página consulta el estado
ACTIVA	Logout	CERRADA	`SessionClosed` → `RETIRAR`; Accounting-Stop
ACTIVA	Callback de inactividad o tope (app G3)	CERRADA	Las reglas ya fueron retiradas; Accounting-Stop
ACTIVA	`MAC_Moved` o cookie usada desde otra IP	INVALIDADA	`SessionClosed(motivo)` → `RETIRAR`; el puerto vuelve a BASE
ACTIVA	`DELETE /sesiones/{id}` (I7, rol TI)	CERRADA	Igual que el logout, con el actor auditado

Tabla 6.1 — Máquina de estados de la sesión en el IAM. Todas las salidas de ACTIVA convergen en un retiro por cookie.

6.5 RADIUS y LDAP (I4)

Mensaje	Atributos	Para qué
Access-Request	User-Name, User-Password (PAP cifrado con el secreto compartido), NAS-IP = IAM, Calling-Station-Id = MAC, Message-Authenticator	FreeRADIUS hace un bind LDAP como el usuario (`Auth-Type LDAP`)
Access-Accept	`Class` = `employeeType` del directorio (p. ej. `ACAD.DOCENTE`)	El rol para el RBAC
Access-Reject	— (con `reject_delay` = 1 s)	Frena la fuerza bruta: cada rechazo se retrasa `reject_delay`
Accounting-Request	Acct-Status-Type Start/Interim(300 s)/Stop, Acct-Session-Id = `sesion_id`	Trazabilidad AAA correlacionable con la auditoría

La configuración de FreeRADIUS es: módulo `ldap` con `base_dn = ou=comunidad,dc=g3,dc=pucp,dc=lab`, `update { reply:Class := 'employeeType' }` y `Auth-Type := LDAP` cuando hay `User-Password`. En el prototipo LDAP va en claro por TCP 389 dentro de VLAN 20; en un despliegue real debe ser LDAPS 636 o StartTLS (hueco H6).

7 M05 Políticas

Es el único punto de decisión. Combina **RBAC** (el `Class` de RADIUS o el rol del operador fija el perfil) con **ABAC** (condiciones sobre dispositivo, ubicación, horario, segundo factor y estado de seguridad que pueden negar). No escribe reglas: resuelve el alcance y lo ordena por I2.

7.1 Condiciones ABAC

Condición	Aplica a	Fuente del atributo	Si falla
`CondicionClaseConocida`	Todos	`Class` ∈ catálogo de `bd_politicas.perfiles`	DENY `clase_desconocida`
`CondicionEstadoSeguridad`	Todos	MAC fuera del conjunto en mitigación (consume `IncidentOpened`/`MitigationExpired`)	DENY `dispositivo_en_mitigacion`
`CondicionDispositivoHabilitado`	Operadores y elevaciones	Registro: MAC activa, vigente, rol máximo ≥ rol pedido	DENY `dispositivo_no_habilitado`
`CondicionUbicacion`	Operadores	Registro: `switch:puerto` declarados (A2:p6 en el prototipo)	DENY `ubicacion_no_declarada`
`CondicionSegundoFactor`	Operadores y aprobadores	IAM: TOTP verificado en esta sesión	DENY `sin_segundo_factor`
`CondicionHorario`	Operadores y elevaciones	`perfiles.ventana` (06:00–23:30 inicial)	DENY `fuera_de_horario`

Tabla 7.1 — Condiciones evaluadas en orden, con deny por defecto: la primera que falla decide y su motivo se audita.

7.2 Decisión y orden

```
def al_session_opened(ev):                          # consumidor de la cola 'politicas'
    if ya_procesado(ev.evento_id): return             # idempotencia (7 dias de ids)
    sol = Solicitud(ev.clase, ev.mac, ev.switch, ev.puerto, ev.totp_ok, ahora())
    for cond in CONDICIONES:                            # deny por defecto
        ok, motivo = cond(sol, contexto)
        if not ok: return publicar('ProfileRejected', ev.sesion, motivo)
    alcance = ALCANCE[perfil_de(ev.clase)]              # lista (destino, proto, puerto)
    orden = Orden('APLICAR_PERFIL', dispositivo=ev.asociacion, alcance=alcance,
  vigencia=VIGENCIA[ev.clase], cookie=ev.sesion, espejo_c1c2=es_operador(ev.clase))
    r = i2.post('/api/v1/ordenes', orden, timeout=2, failover=[ONOS_1, ONOS_2, ONOS_3])
    if r.status == 201: publicar('ProfileApplied', ev.sesion, reglas=r.reglas)
    else:               publicar('ProfileRejected', ev.sesion, 'instalacion:' + r.codigo)
```

Sobre el failover I2: si un nodo no responde en 2 s, se reintenta **la misma orden** en el siguiente. Como la orden es idempotente por cookie, el reintento no duplica reglas aunque el primer nodo sí la hubiera instalado.

7.3 Elevación temporal

Estado	Evento	Siguiente	Efecto
—	`ElevationRequested` (IAM)	SOLICITADA	Alerta en la Consola
SOLICITADA	`POST /elevaciones/{id}/aprobar` (I7, Admin con TOTP) y ABAC ✓	APROBADA	Orden `APLICAR_PERFIL` ELEVADO, TTL ≤ 3600 s, cookie `ELE`
APROBADA	201 APLICADA	ACTIVA	`ElevationGranted` (eco)
SOLICITADA	Rechazo o 15 min sin respuesta	RECHAZADA	Auditoría
ACTIVA	Callback FLOW_REMOVED (hard_timeout)	VENCIDA	`ElevationExpired`; la sesión ACADÉMICA sigue intacta
ACTIVA	Revocación (I7) o cierre de la sesión base	REVOCADA	`RETIRAR` cookie `ELE`

La elevación **no toca** las entradas de la sesión: agrega dos p200 con su propia cookie. Al vencer, el switch las borra solo y el usuario conserva su perfil ACADÉMICO. Si se cierra la sesión base, Políticas retira también sus elevaciones, porque una elevación no puede sobrevivir a la sesión de la que cuelga.

8 M07 Registro

Mantiene el catálogo de **dispositivos privilegiados**: los de operadores y los de académicos que pueden pedir elevación. Los equipos académicos comunes no se registran. Registrar **no concede**: habilita el intento. Expone por I7 `GET/POST /dispositivos`, `GET /dispositivos/{mac}` y `GET /dispositivos/{mac}/historial`, y Políticas consulta `GET /dispositivos/{mac}` con un caché local de 60 s que se invalida con `DeviceRevoked`.

```
Dispositivo { mac, tipo, titular, rol_maximo: OP.TI|OP.ADMIN|OP.SUPERADMIN|ACAD.ELEGIBLE,
  puertos_permitidos: ['A2:p6'], estado: ACTIVO|REVOCADO, vigencia_hasta, responsable }
```

9 M01 — App G3 en el clúster ONOS

Es el traductor: recibe el **qué** por I2 y decide el **cómo** (match, prioridad, cookie, timeouts, switches). Corre en los tres nodos. Cualquiera atiende I2, pero la escritura sobre cada switch la hace ONOS a través de su maestro (ROLE_REQUEST). El estado vive en ConsistentMap replicado por Raft, de modo que sobrevive a la caída de un nodo.

9.1 Clases

```
@Path("/api/v1") public class OrdenesResource {
  @POST @Path("ordenes") Response ejecutar(OrdenJson o);  // 201 APLICADA | 400 | 403 | 404 | 409 | 503
  @GET  @Path("asociaciones") Response asociacion(@QueryParam("ip") String ip);
  @GET  @Path("reglas") Response reglas(@QueryParam("cookie") String c);
  @POST @Path("servicios") Response declarar(ServicioJson s); }
@Component(immediate = true) public class TraductorPerfil {
  @Reference FlowRuleService flows; @Reference HostService hosts; @Reference GestorCaminos caminos;
  List<FlowRule> reglas(Orden o, long cookie) {        // una p200 por elemento del alcance
    for (Destino d : o.alcance()) {
      TrafficSelector s = DefaultTrafficSelector.builder().matchInPort(o.puerto())
          .matchEthSrc(o.mac()).matchEthType(IPV4).matchIPSrc(o.ip().toIpPrefix())
          .matchIPDst(d.prefijo()).matchIPProtocol(d.proto()).matchTcpDst(d.puerto()).build();
      out.add(DefaultFlowRule.builder().forDevice(o.switchId()).withSelector(s)
          .withTreatment(caminos.salida(o.switchId(), d.prefijo())).withPriority(200)
          .withCookie(cookie).makeTemporary(o.tope()).fromApp(appId).build()); }
    return out; } }
public class RegistroCookies { ConsistentMap<String, Long> g3Cookies; long asignar(String texto); }
```

9.2 Aplicar con confirmación (BARRIER)

```
CompletableFuture<Resultado> aplicar(Orden o) {
  long c = cookies.asignar(o.cookie());  // idempotente: misma cookie, mismo valor
  if (yaInstalada(c, o)) return completed(APLICADA);      // reaplicar no duplica
  List<FlowRule> rs = traductor.reglas(o, c);
  CompletableFuture<Resultado> f = new CompletableFuture<>();
  flows.apply(FlowRuleOperations.builder().add(rs)        // un solo lote por orden
     .build(new FlowRuleOperationsContext() {
  public void onSuccess(FlowRuleOperations ops) { f.complete(APLICADA(rs)); }  // = BARRIER_REPLY
  public void onError(FlowRuleOperations ops)  { f.complete(FALLIDA("TABLE_FULL u otro")); } }));
  return f.orTimeout(1500, MILLISECONDS);                  // -> 503, el cliente reintenta
}
```

Siguiendo el patrón de lote del Laboratorio 4, el 201 se emite en `onSuccess`, es decir, cuando el switch ya confirmó la regla (no cuando el controlador solo aceptó la orden). Una tabla llena devuelve error y la sesión se rechaza. **Nunca** se degrada a una regla más amplia.

9.3 Esqueleto, asociaciones y redirección

- `InstaladorEsqueleto`: al conectar un switch instala las entradas genéricas (p150, p140, p130, p100, p30 por puerto de acceso, table-miss). Al aprender un dispositivo por su primer ARP, instala sus p110 y p105 y publica `DeviceConnected`.
- `AsociacionesService`: mantiene IP → (MAC, switch, puerto) a partir de HostService y lo replica en `g3-asociaciones`. Si una MAC aparece en otro puerto, emite `MAC_Moved`.

- `RedireccionPortal`: ante un PACKET_IN de p110 instala por 30 s el par de reglas de reescritura (`ipv4_dst` → 10.0.0.5:80 en el ingreso y la inversa en C1/C2) y devuelve el SYN con PACKET_OUT. El mecanismo final depende de V3/V4 (hueco H3).

9.4 Inactividad y vencimientos

```
@Scheduled cada 60 s:  for (sesion : cookiesDeTipo(SES))
    long p = flows.getFlowEntriesById(appId).filter(cookie == sesion).sum(FlowEntry::packets);
  if (p == ultimo[sesion]) { inactivo[sesion] += 60; } else { inactivo[sesion] = 0; ultimo[sesion] = p; }
  if (inactivo[sesion] >= idle[sesion]) { retirar(sesion); callback(IAM, sesion, "inactividad"); }
RULE_REMOVED (hard_timeout) de una cookie ELE/SES -> retirar el resto de sus entradas -> callback
callback = POST https://<suscriptor>/internal/v1/vencimientos {cookie, motivo, ts}  (IAM: SES, Politicas: ELE)
```

ONOS ya consulta las estadísticas de flujo de cada switch periódicamente (por defecto cada 5 s), así que medir la inactividad por cookie no agrega tráfico OpenFlow. La resolución es de 60 s, suficiente frente a vigencias de 15 y 30 minutos.

10 Contratos

10.1 I2 — orden APLICAR_PERFIL

```
POST https://10.0.0.20:8181/api/v1/ordenes  (Authorization: Bearer <clave de servicio de politicas>)
{ "orden": "APLICAR_PERFIL",
  "perfil": "ACAD.DOCENTE",
  "dispositivo": { "mac": "52:54:00:01:10", "ip": "10.1.0.10", "switch": "A1", "puerto": "p20" },
  "alcance": [ {"destino": "10.2.0.10/32", "proto": "tcp", "puerto": 443},
               {"destino": "10.3.0.0/24",  "proto": "any"} ],
  "vigencia": { "idle_s": 1800, "tope_s": 43200 },
  "espejo_c1c2": false,
  "contexto": { "cookie": "SES-000123", "solicitante": "politicas", "motivo": "login-academico" } }
-> 201 { "estado": "APLICADA", "reglas": 2, "switches": ["A1"], "confirmada_en": "2026-10-11T14:03:22.418Z" }
```

Los errores siguen G-01 §2: `400` esquema, `401` clave, `403` el solicitante no es Políticas, `404` asociación desconocida, `409` cookie reutilizada con otro contenido, `503` sin confirmación a tiempo (se reintenta la misma orden). `RETIRAR` es `{"orden":"RETIRAR","contexto":{"cookie":"SES-000123"}}` → `200 RETIRADA`.

10.2 Eventos de R1/R2 (I5)

Evento	Payload	Productor → consumidores
`DeviceConnected`	mac, ip, switch, puerto	App G3 → Registro, Monitor, Auditoría
`SessionOpened`	sesion, usuario, clase, asociacion, totp_ok, vigencia	IAM → Políticas, Auditoría
`ProfileApplied` / `ProfileRejected`	sesion, perfil, reglas / motivo	Políticas → IAM, Auditoría, Consola
`SessionClosed`	sesion, motivo (logout, inactividad, tope, mac_moved, revocada)	IAM → Políticas, Auditoría
`LoginFailed`	usuario, ip, mac, motivo, intento_n	IAM → Auditoría, Monitor (insumo de R3/R4 para fuerza bruta)
`ElevationRequested` / `Granted` / `Expired`	elev, solicitante, recurso, ttl, aprobador / motivo	IAM o Políticas → Consola, Auditoría
`DeviceRegistered` / `DeviceRevoked`	mac, titular, rol_maximo, responsable	Registro → Políticas, Auditoría, Consola
`MAC_Moved`	mac, ubicacion_anterior, ubicacion_nueva	App G3 / Monitor → IAM, Políticas, Incidentes, Auditoría

Tabla 10.1 — Envoltura común de G-02 (`evento_id` UUIDv7, `emisor`, `timestamp` UTC, `correlacion`, `payload`). El IAM se suscribe a `ProfileApplied` y `ProfileRejected`.

11 Modelo de datos

Base / mapa	Tabla o clave	Columnas principales	Dueño
`bd_iam`	`sesiones`	sesion_id PK · usuario · clase · mac · switch_puerto · ip · idle_s · tope_s · estado · motivo_cierre · creada_en · ultima_actividad	IAM

Base / mapa	Tabla o clave	Columnas principales	Dueño
`bd_iam`	`operadores` · `provision_totp`	usuario PK · hash_bcrypt · totp_secreto (cifrado) · rol · estado	IAM
`bd_iam`	`intentos_fallidos`	(ip, usuario) · n · ventana_desde	IAM
`bd_politicas`	`perfiles`	perfil PK · idle_s · tope_s · alcance (JSON) · elegible_elevacion · ventana	Políticas
`bd_politicas`	`elevaciones`	elev_id PK · sesion_id · solicitante · recurso · ttl_s · estado · aprobador · ts	Políticas
`bd_politicas`	`decisiones`	sesion_id · veredicto · motivo · condiciones evaluadas · latencia_ms	Políticas
`bd_registro`	`dispositivos` · `historial_dispositivo`	mac PK · tipo · titular · rol_maximo · puertos · estado · vigencia_hasta · responsable	Registro
Atomix (ONOS)	`g3-cookies`	texto ↔ valor de 64 bits · orden original · switches	App G3
Atomix (ONOS)	`g3-asociaciones` · `g3-servicios` · `g3-suscripciones`	ip → (mac, switch, puerto) · servicios declarados · URL de callback por tipo de cookie	App G3

No hay JOIN entre bases: la correlación viaja en los eventos (`correlacion.sesion`, `correlacion.dispositivo`). **El plano de datos manda**: tras una caída, el IAM y Políticas reconcilian contra `GET /api/v1/reglas?cookie=…` y cierran cualquier sesión o elevación que ya no tenga reglas vivas.

12 Secuencias con protocolo por salto

12.1 CU-01 — conexión y login de un académico

#	De → a	Mensaje	Protocolo
1	Equipo → A1	ARP del gateway / DHCP DISCOVER	Ethernet · UDP 67/68
2	A1 → ONOS (maestro)	PACKET_IN (p130 ARP o p30)	OpenFlow 1.3 · TCP 6653 por `ma1`
3	ONOS → A1	FLOW_MOD esqueleto del dispositivo (p110, p105) + BARRIER; `DeviceConnected`	OpenFlow 1.3 · AMQP
4	Equipo → portal	HTTP :80 → p110 → reescritura → `302` → `GET /login`	HTTP 80 → HTTPS 443
5	Equipo → Portal → IAM	`POST /login {usuario, clave}`	HTTPS · REST/JSON (I3)
6–7	IAM → FreeRADIUS → LDAP	Access-Request → bind → Access-Accept (`Class`)	RADIUS UDP 1812 → LDAP TCP 389 (I4)
8	IAM	(operador) bcrypt + TOTP local	—
9	IAM → ONOS	`GET /api/v1/asociaciones?ip=10.1.0.10`	REST/HTTPS · TCP 8181 (I2)
10–11	IAM → RabbitMQ → Políticas	`SessionOpened`	AMQP 0-9-1 · TCP 5672 (I5)
12	Políticas	ABAC → perfil efectivo y alcance	—
13	Políticas → ONOS	`POST /api/v1/ordenes APLICAR_PERFIL`	REST/HTTPS · TCP 8181 (I2)
14–15	ONOS ↔ A1	FLOW_MOD p200 ×2 + BARRIER_REQUEST / REPLY	OpenFlow 1.3
16	ONOS → Políticas	`201 APLICADA`	HTTP 201 · JSON
17	Políticas → IAM, Auditoría, Consola	`ProfileApplied`	AMQP (I5)
18	IAM → Portal → equipo	`200 sesión activa` + cookie opaca	HTTPS 443

Tabla 12.1 — Corresponde a la Fig. 10 de la v2. En el paso 17 el IAM consume el eco `ProfileApplied` antes de responder.

12.2 CU-04 — elevación al servidor privilegiado

#	De → a	Mensaje	Protocolo
1–2	Usuario → Portal → IAM	`POST /elevacion {recurso: 10.2.0.11, motivo}`	HTTPS 443 · REST (I3)
3	IAM → Registro	`GET /dispositivos/{mac}`: ¿registrado y elegible?	REST/HTTPS (I7)
4–5	IAM → RabbitMQ → Consola	`ElevationRequested` → alerta en vivo	AMQP (I5)
6	Admin (Consola) → Políticas	`POST /elevaciones/{id}/aprobar` (sesión Admin + TOTP)	REST/HTTPS (I7)

#	De → a	Mensaje	Protocolo
7	Políticas	ABAC: rol ∧ recurso ∧ horario ∧ estado de seguridad	—
8	Políticas → ONOS	`APLICAR_PERFIL ELEVADO`, TTL 3600 s, cookie `ELE-…`	REST/HTTPS · TCP 8181 (I2)
9–10	ONOS ↔ A1	FLOW_MOD p200 ×2 (`hard=3600`, `SEND_FLOW_REM`) + BARRIER	OpenFlow 1.3
11–12	ONOS → Políticas → bus	`201` → `ElevationGranted`	HTTP 201 · AMQP
13	Usuario → srv .11	SSH 22 / HTTPS 443: coincide p200 en A1 → D1/D2 → NC1/NC2 → C1	Datos, sin PACKET_IN
14–16	A1 → ONOS → Políticas → bus	FLOW_REMOVED → callback → `ElevationExpired`	OpenFlow · REST · AMQP

Tabla 12.2 — Corresponde a la Fig. 12 de la v2.

13 Configuración

Parámetro	Valor inicial	Dónde	Se calibra observando
Vigencias ACAD / OP / ELEVADO	30 min · 12 h / 15 min · 8 h / 1 h	`bd_politicas.perfiles`	Re-logins observados
Resolución de inactividad (app G3)	60 s	Propiedad del componente ONOS	Error de expiración
Espera del eco en el IAM	2,5 s (luego 202)	IAM	Latencia de login hasta la regla activa
Timeout I2 por nodo / reintento	2 s / siguiente nodo	Cliente I2 común	Latencia I2 por nodo (M4)
Timeout RADIUS / reintentos	3 s / 2	IAM	Latencia I4
`reject_delay` de FreeRADIUS	1 s	`radiusd.conf`	Retraso por rechazo
Limitador de intentos fallidos	5 por (IP, usuario) en 5 min → 429 durante 5 min	IAM	Tasa de bloqueo
Coste de bcrypt (operadores)	10	IAM	Tiempo de verificación por login
TOTP	RFC 6238, 30 s, ventana ±1	IAM	—
Meters `m_redir` / `m_miss` / `m_arp`	20 / 50 / 50 pps por switch	App G3	PACKET_IN/s en ráfaga
Ventana horaria de operadores y elevaciones	06:00–23:30	`bd_politicas`	—

Lo constante del diseño (prioridades, esquema de cookies, separación entre política y caminos) no es configurable. Todo cambio de parámetro entra por I7 y queda auditado con su autor.

14 Fallos

Falla	Sigue funcionando	Se pierde	Diseño
Cae un nodo ONOS	Todo: sus switches piden MASTER a los otros dos; reglas intactas	Nada perceptible; las órdenes en curso reintentan en otro nodo	Raft 2 de 3; I2 con failover; idempotencia por cookie
Caen 2 nodos ONOS	Reglas instaladas y sesiones vigentes	Logins y elevaciones nuevas (sin quórum)	Degradación declarada; el IAM responde 503
Cae FreeRADIUS o LDAP	Sesiones vigentes; operadores (repositorio propio)	Logins nuevos de la comunidad	Fail-closed: 503 en el portal
Cae RabbitMQ	Reglas instaladas; el IAM no completa logins	Logins nuevos	Buffer local de 10 000 mensajes; huecos declarados
Cae Políticas	Lo instalado	Logins, elevaciones y retiros por logout	Al volver consume su cola durable; los topes siguen en el switch
Tabla del switch llena	Sesiones existentes	Nuevas sesiones en ese switch	`onError` → `ProfileRejected`; nunca una regla más amplia
Cae el enlace A1–D1	Todo, por D2	Conmutación inmediata con grupos FF; con recálculo, tras la reconvergencia	Pruebas RD1–RD4 de la v2

15 Trazabilidad con la rúbrica

La tabla indica, para cada criterio de R1 y R2, qué elemento de este diseño lo implementa y qué punto del sistema permite medirlo después. Este documento define los puntos de medición; no contiene resultados.

Criterio	Clase o función	Regla o dato	Punto de medición disponible
R1 · roles y permisos mínimos	Escalera de perfiles, `ALCANCE`, `CONDICIONES`	Tablas 4.1, 4.3 y 7.1	Matriz perfil × recurso contra las entradas p200 instaladas
R1 · flexibilidad	`bd_politicas.perfiles` vía I7; alcance en la orden	Sin redeploy de la app G3	Cambio de alcance de un perfil sin reiniciar servicios
R1 · administración amigable	Consola (M10): sesiones, aprobaciones, registro	I7	—
R1 · tasa de bloqueo	`VerificadorComunidad`, p105, p30	Access-Reject; deny por dispositivo	Intentos no autorizados frente a contadores de p105 y rechazos RADIUS
R1 · latencia de autenticación	Cadena de la Tabla 12.1	Marcas de tiempo `/login` → `ProfileApplied`	Eventos con `timestamp` y `correlacion.sesion`
R1 · escalabilidad (logins/s)	IAM, I4, I2, BARRIER	Un lote y un BARRIER por orden	Carga concurrente sobre `/login`, RADIUS e I2
R2 · definición y nivel de protección	Tabla 4.2	General / privilegiado / crítico	—
R2 · precisión de la restricción	p200 por destino + p105	Matriz 4.3	Contadores de p200 y p105 por cookie
R2 · sesiones concurrentes	`TraductorPerfil`, lote por orden	Entradas por sesión	`GET /api/v1/reglas?cookie=…` y conteo de entradas
R2 · sobrecarga (TCAM, CPU, memoria)	Tabla 5.1	Entradas por switch según sesiones y equipos	Estadísticas de tabla del switch (MULTIPART) y uso de recursos de ONOS

16 Huecos y decisiones pendientes

#	Hueco	Propuesta	Cierra
H1	V1: ¿PicOS acepta multi-tabla y `write_metadata` con máscara?	Probar `TABLE_FEATURES` en VNRT; si sí, activar la variante §5.5	Vincenzo · sem. 11
H2	Capacidad real de TCAM del Pica8 (valor aún no verificado)	Llenado por lotes hasta `OFPFMFC_TABLE_FULL`	Vincenzo · LAB-06
H3	Mecanismo de redirección al portal (reescritura vs. PACKET_OUT)	Medir V3/V4 y elegir el de menor PACKET_IN/s	Piero
H4	Tasa de FLOW_MOD/s y latencia del BARRIER del Pica8	Medir en el laboratorio; los valores de partida son supuestos sin verificar	LAB-06
H5	Elegibilidad por subperfil (¿alumno elegible a .11?)	Inicial: no; validar con el coach	Rodrigo · martes
H6	LDAP en claro (TCP 389) dentro de VLAN 20	Prototipo: aceptado; despliegue: LDAPS 636 / StartTLS	Documentado
H7	MAC + IP suplantadas en el mismo puerto (NIC-B compartida por los 16 académicos)	No es detectable por el pipeline; se declara como limitación (en el campus real cada puerto tiene un equipo)	Documentado
H8	Agregación de caminos en el campus (/32 por host no escala en el núcleo)	Prefijo por switch de acceso (/26)	HLD de referencia
H9	Clúster de 2 o de 3 nodos («dos clústeres» del coach)	3 nodos (Raft necesita mayoría); confirmar con el coach	Rodrigo
H10	IPs de ONOS (.20/.30/.40) y memoria del servidor de 32 GB	Confirmar con Vincenzo	Vincenzo
H11	Alinear la v2 con este diseño: pág. 9 (prioridades), Fig. 15 (cookies) y Fig. 10 paso 17 (eco)	Actualizar antes de exponer	Piero

17 Declaración de uso de inteligencia artificial

Conforme a los lineamientos del curso, el Grupo 3 declara que usó inteligencia artificial generativa (Claude, de Anthropic) para contrastar el diseño con la propuesta v2 y con el repositorio, y redactar este documento. Las decisiones y el contenido fueron revisados por el equipo, que responde por ellos.